MyHarvia Cloud API Specification

GRAPHQL AWS TYPESCRIPT

> Harvia API — Initial Release, v 0.5.0 - 031125
> We're publishing the first public release of the Harvia API. This is an early developer release: we invite you to explore, integrate, and provide feedback.
> The API will continue to evolve as we add capabilities and add schemas. We are also evaluating alternative API approaches — including MQTT-based communication, AI-compatible data interfaces, and other modern integration models. Our goal is to ensure long-term interoperability, efficiency, and ease of use across both embedded and cloud environments. We strive to maintain backward compatibility; when changes are necessary, we will announce them in advance and provide clear migration guidance through our documentation and changelog.

On this page

- Overview
- Common GraphQL Scalar Types
- GraphQL Operations
  - Device Service
  - Data Service
  - Events Service
- GraphQL Complete Example
  - JavaScript/fetch
  - Python/requests
  - cURL
- REST API Operations
  - Authentication
  - API Endpoint Configuration
  - Getting the Endpoints Programmatically
  - JWT Token Authentication
    - Obtaining JWT Token
  - Refreshing JWT Token
  - Revoking Refresh Token

Overview

This document covers the MyHarvia Cloud API specification with 3 services and their APIs, authentication setup,

and configuration.

- **Authentication** - REST API authentication and JWT token management via the public REST API
- **API Services**:
  - **Data Service** - Device measurements and session data (GraphQL + REST subset)
  - **Device Service** - Device management and control (GraphQL + REST subset)
  - **Events Service** - Event monitoring and notifications (GraphQL + REST subset)

API Types:

- **GraphQL** - The full API surface for queries, mutations, and subscriptions (available in all services)
- **REST API** - A subset of common operations exposed as REST endpoints (available in Data, Device & Events Services)

Available Endpoints:

- **GraphQL HTTPS endpoint** - For standard GraphQL queries and mutations (all services)
- **GraphQL WebSocket endpoint** - For real-time GraphQL subscriptions (all services)
- **GraphQL Schema URL** - For downloading the GraphQL schema (all services)
- **REST API endpoints** - Standard HTTP REST endpoints for the REST subset (Data, Device & Events Services)

MCP server:

- Control and monitor Harvia sauna devices through natural language with Claude or any MCP-compatible client. See the setup guide and GitHub repo

🔤 Common GraphQL Scalar Types

These common GraphQL scalar types appear across the service schemas and examples.

Type	Description	Exampl
`String`	Standard string type	`"device`
`Int`	Integer number	`42`
`Float`	Floating-point number	`3.14`
`ID`	Unique identifier	`"abc123`
`AWSDateTime`	AWS datetime scalar (ISO 8601 format)	`"2025-0`
`AWSJSON`	AWS JSON scalar (JSON as string)	`"{\"tem`

🔵 GraphQL Operations

GraphQL is the primary and complete

GraphQL is the primary and complete API surface across the services. REST endpoints are provided only for a subset of common operations.

🔧 Device Service

Operation Type	Description
Queries	Device lookup, search, and state retrieval
Mutations	Device control, configuration changes, and management actions
Subscriptions	Real-time device events and state updates

📊 Data Service

Operation Type	Description
Queries	Measurements, telemetry history, and generated reports
Subscriptions	Real-time measurement and session data updates

📣 Events Service

Operation Type	Description
	Event and

Queries	notification retrieval
Mutations	Event state management where supported
Subscriptions	Real-time event and notification streams

📦 GraphQL Complete Example

This end-to-end example shows a GraphQL flow: fetch configuration, authenticate through the REST API, then call a GraphQL service endpoint.

🟨 JavaScript/fetch

```
async function getApiConfiguration()
  const response = await fetch("https
  const { endpoints } = await respons
  const restApiBaseUrl = endpoints.Re

  return {
    restApiBaseUrl,
    graphql: endpoints.GraphQL,
  };
}

async function signInAndGetIdToken(us
  try {
    const config = await getApiConfig
    const response = await fetch(`${c
      method: "POST",
      headers: { "Content-Type": "app
      body: JSON.stringify({ username
    });

    if (!response.ok) {
      const error = await response.js
      throw new Error(error.message |
    }

    const tokens = await response.jso
    return {
      idToken: tokens.idToken,
      accessToken: tokens.accessToken
```

```
      refreshToken: tokens.refreshTok
      expiresIn: tokens.expiresIn,
    };
  } catch (error) {
    console.error("Authentication err
    throw error;
  }
}

// Perform a GraphQL POST to a servic
async function makeGraphQLRequest() {
  const config = await getApiConfigur
  const tokens = await signInAndGetId
  console.log("ID Token:", tokens.idT

  await fetch(config.graphql.data.htt
    method: "POST",
    headers: {
      "Content-Type": "application/js
      "Authorization": `Bearer ${toke
    },
    body: JSON.stringify({ query: "<y
  });
  // See individual service docs (dat
}

makeGraphQLRequest();
```

Python/requests

```
import requests

def get_api_configuration():
    response = requests.get("https://
    endpoints = response.json()["endp
    rest_api_base_url = endpoints["Re

    return {
        "rest_api_base_url": rest_api
        "graphql": endpoints["GraphQL
    }

def sign_in_and_get_id_token(username
    config = get_api_configuration()
    response = requests.post(
        f"{config['rest_api_base_url'
        headers={"Content-Type": "app
        json={"username": username, "
    )

    if not response.ok:
        error = response.json()
        raise Exception(error.get("me

    tokens = response.json()
    return {
        "id_token": tokens["idToken"]
        "access_token": tokens["acces
```

```
            access_token": tokens["acces
        "refresh_token": tokens["refr
        "expires_in": tokens["expires
    }

# Perform a GraphQL POST to a service
config = get_api_configuration()
tokens = sign_in_and_get_id_token("yo
print(f"ID Token: {tokens['id_token']

requests.post(
    config["graphql"]["data"]["https"
    headers={
        "Content-Type": "application/
        "Authorization": f"Bearer {to
    },
    json={"query": "<your GraphQL que
)
# See individual service docs (data/d
```

🔧 cURL

```
# 1. Get REST API base URL and GraphQ
ENDPOINTS=$(curl -sS "https://api.har
REST_API_BASE=$(echo "$ENDPOINTS" | j
GRAPHQL_DATA=$(echo "$ENDPOINTS" | jq

# 2. Authenticate and get tokens
TOKEN_RESPONSE=$(curl -sS -H "Content
  -X POST "$REST_API_BASE/auth/token"
  --data '{"username":"your-username"

ID_TOKEN=$(echo "$TOKEN_RESPONSE" | j
echo "ID Token: $ID_TOKEN"

# 3. Make a GraphQL request
curl -sS -H "Content-Type: applicatio
  -H "Authorization: Bearer $ID_TOKEN
  -X POST "$GRAPHQL_DATA" \
  --data '{"query":"<your GraphQL que

# See individual service docs (data/d
```

📋 REST API Operations

REST API operations cover a smaller subset of common integration flows. Authentication and endpoint discovery are handled through the public REST API.

🔐 Authentication

Endpoint	Method	Description
`/auth/token`	`POST`	Username & password-based login, returns tokens (valid for 1 hour) for API access
`/auth/refresh`	`POST`	Refresh tokens to extend API access
`/auth/revoke`	`POST`	Revoke refresh tokens

🔧 Device Service

Endpoint	Method	Description
`/devices`	`GET`	List user's devices
`/devices/command`	`POST`	Send command to a device
`/devices/state`	`GET`	Get device state (shadow)
`/devices/target`	`PATCH`	Set target temperature & humidity
`/devices/profile`	`PATCH`	Change device profile

📊 Data Service

Endpoint	Method	Description
`/data/latest-data`	`GET`	Get latest telemetry data
`/data/telemetry-history`	`GET`	Get telemetry history for a time range
`/data/sessions`	`GET`	Get device sessions for a time range

📣 Events Service

Endpoint	Method	Description
`/events/events`	`GET`	Get device events for a time range

🔐 Authentication

The APIs use **AWS Cognito** for authentication via JWT tokens. All API endpoints require JWT token authentication. Authentication is handled through the **public REST API** which provides token endpoints.

Authorization Type	Description	Usa
JWT Token	Cognito authentication via REST API	**All acc**

Fetch API configuration first to obtain

the REST API base URL and GraphQL endpoints. Use the REST API `/auth/token` endpoint to obtain JWT tokens.

🌐 API Endpoint Configuration

Endpoint: https://api.harvia.io/endpoints

This endpoint returns the values required for authentication and service requests.

API Response Example:


```
{
  "endpoints": {
    "DataPipeline": {
      "TelemetryIngestionStream": "ha
      "TelemetryIngestionEndpoint": "
      "TelemetryIngestionRole": "harv
    },
    "Config": {
      "RequiredClientVersion": "0.0.0
      "Region": "eu-central-1",
      "Purchases": {
        "consumablesEnabledCountries"
          "AT",
          "AUT",
          "BE",
          "BEL",
          "BG",
          "BGR",
          "HR",
          "HRV",
          "CY",
          "CYP",
          "CZ",
          "CZE",
          "DK",
          "DNK",
          "EE",
          "EST",
          "FI",
          "FIN",
          "FR",
          "FRA",
          "DE",
          "DEU",
          "GR",
          "GRC",
          "HU",
          "HUN",
```

```
            "IE",
            "IRL",
            "IT",
            "ITA",
            "LV",
            "LVA",
            "LT",
            "LTU",
            "LU",
            "LUX",
            "MT",
            "MLT",
            "NL",
            "NLD",
            "PL",
            "POL",
            "PT",
            "PRT",
            "RO",
            "ROU",
            "SK",
            "SVK",
            "SI",
            "SVN",
            "ES",
            "ESP",
            "SE",
            "SWE",
            "NO",
            "NOR",
            "GB",
            "GBR",
            "US",
            "USA",
            "CA",
            "CAN",
            "AU",
            "AUS",
            "NZ",
            "NZL",
            "MX",
            "MEX",
            "AE",
            "ARE",
            "SA",
            "SAU",
            "EG",
            "EGY",
            "ZA",
            "ZAF",
            "VN",
            "VNM",
            "TH",
            "THA",
            "SG",
            "SGP",
            "MY",
            "MYS",
            "ID",
            "IDN",
```

```
      "GE",
      "GEO",
      "AM",
      "ARM"
    ],
    "subscriptionsEnabledCountrie
  },
  "UserPoolId": "eu-central-1_PYo
  "PartnerOrganizationId": "ORG/p
  "PinpointAnalyticsProjectId": "
  "IoTCoreCredentialsEndpoint": "
  "Urls": {
    "harviaWeb": {
      "privacyPolicy": {
        "en": "https://www.harvia
      },
      "termsOfService": {
        "jp": "https://www.harvia
        "de": "https://www.harvia
        "en-US": "https://www.har
        "et": "https://www.harvia
        "fi": "https://www.harvia
        "en": "https://www.harvia
        "sv": "https://www.harvia
      },
      "support": {
        "en": "https://www.harvia
      },
      "imprint": {
        "en": ""
      },
      "termsOfUse": {
        "en": "https://www.harvia
      },
      "privacyNotice": {
        "en": "https://www.harvia
        "fi": "https://www.harvia
        "de": "https://www.harvia
        "jp": "https://harvia.jp/
        "sv": "https://www.harvia
        "et": "https://www.harvia
        "en-US": "https://www.har
      }
    },
    "eosWeb": {
      "support": {
        "en": "https://www.eos-sa
      },
      "privacyPolicy": {
        "en": "https://www.eos-sa
      },
      "termsOfService": {
        "en": "https://www.eos-sa
        "de": "https://www.eos-sa
      },
      "termsOfUse": {
        "en": "http://www.eos-sau
      },
      "imprint": {
        "en": "https://www.eos-sa
```

```
      },
      "privacyNotice": {
        "en": "https://www.eos-sa
        "de": "https://www.eos-sa
      }
    },
    "privacyPolicy": "https://www
    "harvia": {
      "eula": {
        "en": "https://cdn.harvia
      },
      "termsOfService": {
        "et": "https://www.harvia
        "fi": "https://www.harvia
        "de": "https://www.harvia
        "sv": "https://www.harvia
        "en": "https://www.harvia
        "en-US": "https://www.har
        "jp": "https://www.harvia
      },
      "privacyPolicy": {
        "en": "https://cdn.harvia
      },
      "termsOfUse": {
        "en": "https://cdn.harvia
      },
      "privacyNotice": {
        "en": "https://www.harvia
        "de": "https://www.harvia
        "en-US": "https://www.har
        "fi": "https://www.harvia
        "jp": "https://harvia.jp/
        "et": "https://www.harvia
        "sv": "https://www.harvia
      }
    },
    "termsOfUse": "https://www.ha
    "eos": {
      "privacyPolicy": {
        "en": "https://cdn.harvia
      },
      "termsOfUse": {
        "en": "https://cdn.harvia
      },
      "privacyNotice": {
        "de": "https://www.eos-sa
        "en": "https://www.eos-sa
      },
      "eula": {
        "en": "https://cdn.harvia
      },
      "termsOfService": {
        "en": "https://www.eos-sa
        "de": "https://www.eos-sa
      }
    }
  },
  "Clients": [
    {
      "name": "harvia-eos-generic
```

```
        "id": "1cq6ttro3ug8u0h2qbu9
      },
      {
        "id": "24emhb2mm0v4sscqhbde
        "name": "harvia-eos-generic
      },
      {
        "name": "harvia-eos-generic
        "id": "2m6409shai9f9isotj1j
      },
      {
        "id": "u3rf9f4n4bec1eouni4n
        "name": "harvia-eos-generic
      }
    ],
    "IotCoreEndpoint": "ab4fm8yv2cf
    "AnalyticsDashboardId": "afdd1e
    "AuthType": "AMAZON_COGNITO_USE
    "PinpointAnalyticsLevel": "medi
    "IdentityPoolId": "eu-central-1
  },
  "version": "6.73.3",
  "GraphQL": {
    "notify": {
      "schemaUrl": "https://appsync
      "wss": "wss://mjuf5vdr6jdjfo7
      "https": "https://mjuf5vdr6jd
    },
    "data-export": {
      "https": "https://62tloxquhfc
      "schemaUrl": "https://appsync
      "wss": "wss://62tloxquhfcovmz
    },
    "data": {
      "schemaUrl": "https://appsync
      "wss": "wss://b6ypjrrojzfuleu
      "https": "https://b6ypjrrojzf
    },
    "stats": {
      "schemaUrl": "https://appsync
      "wss": "wss://2y6n4pgr6nbmddo
      "https": "https://2y6n4pgr6nb
    },
    "users": {
      "https": "https://qizruaso4na
      "schemaUrl": "https://appsync
      "wss": "wss://qizruaso4naexbn
    },
    "events": {
      "https": "https://ykn3dsmrrvc
      "wss": "wss://ykn3dsmrrvc47ln
      "schemaUrl": "https://appsync
    },
    "payment": {
      "https": "https://2bmloen445d
      "schemaUrl": "https://appsync
      "wss": "wss://2bmloen445dojlg
    },
    "device": {
      "wss": "wss://6lhlukqhbzefnha
```

```
        "https": "https://61h1ukqhbze
        "schemaUrl": "https://appsync
      }
    },
    "RestApi": {
      "users": {
        "https": "https://c3jzg90xli.
      },
      "harvia-eos-generics-prod-s2s-g
        "https": "https://5id5f6leic.
      },
      "data": {
        "https": "https://u4830dkpl0.
      },
      "events": {
        "https": "https://3hlcbophzl.
      },
      "notify": {
        "https": "https://c0fayizgfk.
      },
      "device": {
        "https": "https://ap754v98f8.
      },
      "generics": {
        "https": "https://zft3sdx910.
      }
    },
    "LoggingConfig": {
      "LoggingMode": 0
    }
  }
}
```

🌐 Getting the Endpoints Programmatically

You can fetch the endpoints programmatically using a simple HTTP GET request. The response structure matches the API Response Example shown above.

🟧 JavaScript/fetch

```
const response = await fetch("https:/
const data = await response.json();
console.log(data);
// Response:
// {
//   "endpoints": {
//     "RestApi": {
//       "generics": { "https": "http
//       "users": { "https": "https:/
//
```

```
//       ...
//     },
//     "version": "6.41.0",
//     "Config": { ... },
//     "GraphQL": { ... },
//     ...
//   }
// }
```

Python/requests

```
import requests

response = requests.get("https://api.
data = response.json()
print(data)
# Response:
# {
#   "endpoints": {
#     "RestApi": {
#       "generics": { "https": "https
#       "users": { "https": "https://
#       ...
#     },
#     "version": "6.41.0",
#     "Config": { ... },
#     "GraphQL": { ... },
#     ...
#   }
# }
```

cURL

```
curl -sS "https://api.harvia.io/endpo

# Response:
# {
#   "endpoints": {
#     "RestApi": {
#       "generics": { "https": "https
#       "users": { "https": "https://
#       ...
#     },
#     "version": "6.41.0",
#     "Config": { ... },
#     "GraphQL": { ... },
#     ...
#   }
# }
```

JWT Token Authentication

For API access, you need to

For API access, you need to authenticate and obtain a JWT token using the public REST API. The `IdToken` must be included in the `Authorization` header of all your requests.

🔑 Obtaining JWT Token

Use the REST API `/auth/token` endpoint to authenticate and obtain JWT tokens:

🟧 JavaScript/fetch

```
async function getApiConfiguration()
  const response = await fetch("https
  const { endpoints } = await respons
  const restApiBaseUrl = endpoints.Re

  return {
    restApiBaseUrl,
  };
}

async function signInAndGetIdToken(us
  try {
    const config = await getApiConfig
    const response = await fetch(`${c
      method: "POST",
      headers: { "Content-Type": "app
      body: JSON.stringify({ username
    });

    if (!response.ok) {
      const error = await response.js
      throw new Error(error.message |
    }

    const tokens = await response.jso
    return {
      idToken: tokens.idToken,
      accessToken: tokens.accessToken
      refreshToken: tokens.refreshTok
      expiresIn: tokens.expiresIn,
    };
  } catch (error) {
    console.error("Authentication err
    throw error;
  }
}

// Usage
const tokens = await signInAndGetIdTo
console.log("ID Token:", tokens.idTok
```

🐍 Python/requests

```
import requests

def get_api_configuration():
    response = requests.get("https://
    endpoints = response.json()["endp
    rest_api_base_url = endpoints["Re

    return {
        "rest_api_base_url": rest_api
    }

def sign_in_and_get_id_token(username
    config = get_api_configuration()
    response = requests.post(
        f"{config['rest_api_base_url'
        headers={"Content-Type": "app
        json={"username": username, "
    )

    if not response.ok:
        error = response.json()
        raise Exception(error.get("me

    tokens = response.json()
    return {
        "id_token": tokens["idToken"]
        "access_token": tokens["acces
        "refresh_token": tokens["refr
        "expires_in": tokens["expires
    }

# Usage
tokens = sign_in_and_get_id_token("yo
print(f"ID Token: {tokens['id_token']
```

🔧 cURL

```
# First, get the REST API base URL fr
REST_API_BASE=$(curl -sS "https://api

# Authenticate and get tokens
curl -sS -H "Content-Type: applicatio
  -X POST "$REST_API_BASE/auth/token"
  --data '{"username":"your-username"

# The response contains: { "idToken":
```

Note: Save the `idToken` from the response to use in the `Authorization: Bearer <idToken>` header for API requests.

🔄 Refreshing JWT Token

JWT tokens expire after 1 hour. For long-running applications, you'll need to refresh tokens to maintain API access. Use the REST API `/auth/refresh` endpoint to obtain new tokens.

🟨 JavaScript/fetch

```
async function refreshIdToken(refresh
  try {
    const config = await getApiConfig
    const response = await fetch(`${c
      method: "POST",
      headers: { "Content-Type": "app
      body: JSON.stringify({ refreshT
    });

    if (!response.ok) {
      const error = await response.js
      throw new Error(error.message |
    }

    const tokens = await response.jso
    return {
      idToken: tokens.idToken,
      accessToken: tokens.accessToken
      expiresIn: tokens.expiresIn,
    };
  } catch (error) {
    console.error("Token refresh erro
    throw error;
  }
}

// Usage
// After initial sign-in, store the r
const initialTokens = await signInAnd

// Later, when token expires (after ~
const newTokens = await refreshIdToke
// Use newTokens.idToken for API requ
```

Python/requests

```
def refresh_id_token(refresh_token: s
    config = get_api_configuration()
    response = requests.post(
        f"{config['rest_api_base_url'
        headers={"Content-Type": "app
        json={"refreshToken": refresh
    )
```

```
    if not response.ok:
        error = response.json()
        raise Exception(error.get("me

    tokens = response.json()
    return {
        "id_token": tokens["idToken"]
        "access_token": tokens["acces
        "expires_in": tokens["expires
    }

# Usage
# After initial sign-in, store the re
initial_tokens = sign_in_and_get_id_t

# Later, when token expires (after ~1
new_tokens = refresh_id_token(initial
# Use new_tokens["id_token"] for API
```

🔧 cURL

```
# Get REST API base URL (if not alrea
REST_API_BASE=$(curl -sS "https://api

# Refresh tokens using the refresh to
curl -sS -H "Content-Type: applicatio
  -X POST "$REST_API_BASE/auth/refres
  --data '{"refreshToken":"<refreshTo

# The response contains: { "idToken":
```

🔒 Revoking Refresh Token

You can revoke a refresh token to invalidate it and prevent future token refreshes. Use the REST API `/auth/revoke` endpoint.

Note: Revoking a refresh token does not invalidate existing ID tokens; they remain valid until expiry.

🟨 JavaScript/fetch

```
async function revokeRefreshToken(ref
  try {
    const config = await getApiConfig
    const response = await fetch(`${c
      method: "POST",
      headers: { "Content-Type": "app
      body: JSON.stringify({ refreshT
```

```
    });

    if (!response.ok) {
      const error = await response.js
      throw new Error(error.message |
    }

    const result = await response.jso
    return result;
  } catch (error) {
    console.error("Token revocation e
    throw error;
  }
}

// Usage
// After initial sign-in, store the r
const initialTokens = await signInAnd

// Later, revoke the refresh token
await revokeRefreshToken(initialToken
// Returns: { "success": true }
```

Python/requests

```
def revoke_refresh_token(refresh_toke
    config = get_api_configuration()
    response = requests.post(
        f"{config['rest_api_base_url'
        headers={"Content-Type": "app
        json={"refreshToken": refresh
    )

    if not response.ok:
        error = response.json()
        raise Exception(error.get("me

    return response.json()

# Usage
# After initial sign-in, store the re
initial_tokens = sign_in_and_get_id_t

# Later, revoke the refresh token
result = revoke_refresh_token(initial
# Returns: { "success": True }
```

cURL

```
# Get REST API base URL (if not alrea
REST_API_BASE=$(curl -sS "https://api

# Revoke refresh token
curl -sS -H "Content-Type: applicatio
  -X POST "$REST_API_BASE/auth/revoke
    data '{"refreshToken":"<refreshTo
```

```
  --data '{ "refreshToken": "<refresh
# The response contains: { "success":
```

🌎 Configuration Reference: [Harvia Endpoints API]

📝 Note: Always fetch the latest configuration to ensure you're using the current REST API and GraphQL endpoints. The configuration may change over time.
